

Cleft — Project Summary

1. Problem Definition

Group activities such as dining, gatherings, and shared outings often create a common problem: money is spent by several people, but settlement is usually done manually with chat messages, spreadsheet formulas, or memory.

This leads to three main issues:

- Inconsistent item tracking: not every participant knows which expenses belong to whom.
- Manual calculation errors: total and share values are easy to miscalculate.
- Poor accountability: there is no clear record of who paid, who owes, and which items were confirmed.

Cleft is designed to solve this by providing a simple group bill-splitting workflow where users can:

- add a party
- list items and members
- choose split logic
- confirm the final calculation
- view payment history and summary

2. Project Scope

In scope

- Google-based authentication and protected routes
- party creation and date tracking
- item-level cost entries and notes
- participant management
- split modes: ALL and PARTIAL
- calculation of per-person share and payer balance
- final confirmation workflow
- database persistence for users, parties, and histories
- local draft session before final confirmation

Out of scope

- real-time multi-user live synchronization

- recurring expenses or subscription billing
- multi-currency exchange conversion
- push notifications and email reminders
- advanced analytics or export reports
- payment settlement execution (cash transfer, e-wallet integration)

This project is intentionally scoped as a practical full-stack app for portfolio/demo use rather than a production payment platform.

3. Why This Project Structure Was Created

The project is organized to separate responsibilities and keep the system easier to understand, test, and deploy.

Root-level files

- Dockerfile: build the backend runtime for container-based deployment
- docker-compose.yml: run project services together in local development
- .env.example: template for local environment variables

Frontend folder

- frontend/: main web application built with Next.js
- purpose: provide user experience, session handling, item/member summary screens, and client-side split calculations
- reason: this layer is responsible for the interactive user-facing experience and is separated from backend logic for easier deployment and clearer responsibility boundaries

Backend folder

- backend/: NestJS API server with Prisma ORM
- purpose: authenticate users, validate requests, compute and persist party data, and serve app logic through REST endpoints
- reason: API logic should remain separated from UI code so it can be deployed independently and reused by other clients in the future

Supporting folders

- edit_front_end/: prototype or design iteration area for UI experimentation
- scripts/: helper utilities such as JWT generation and PDF export
- uploads/: storage area for uploaded slip images or local attachments

This separation keeps each layer focused on one concern: UI, API, database, and tooling.

4. Data Pipeline

The system data path is designed around a clear flow from user action to persistence.

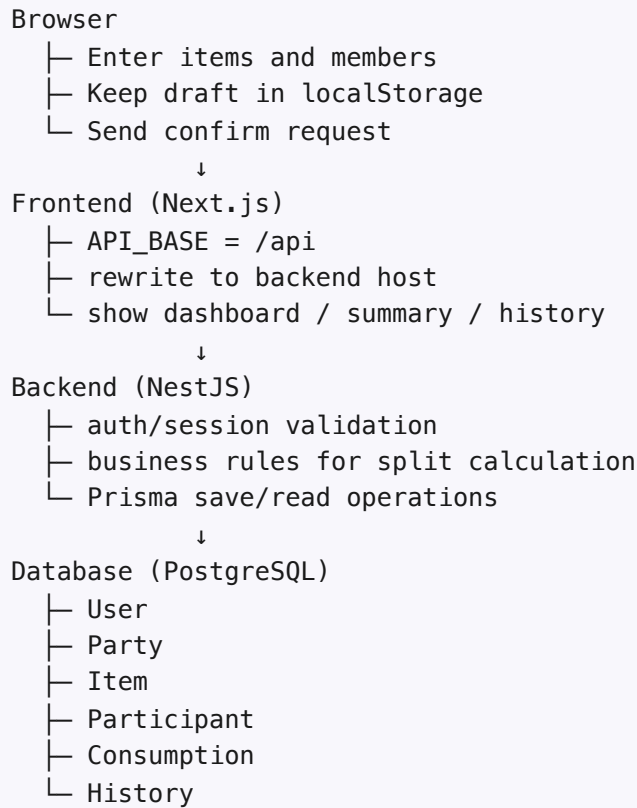
```
User action
  ↓
Next.js frontend
  ↓
/api rewrite via next.config.mjs
  ↓
NestJS backend
  ↓
Validation + auth + business logic
  ↓
Prisma ORM
  ↓
PostgreSQL database
```

Pipeline flow in practice

1. A user opens a page in the frontend and creates or edits a party.
2. Draft data is stored in localStorage before confirmation.
3. When the user confirms, the frontend sends the payload to the backend API.
4. The backend checks JWT/session validity and validates party data.
5. The backend calculates participant shares and resulting totals.
6. Prisma writes records into PostgreSQL.
7. The frontend fetches updated party history and summary data for display.

The server does not trust client-side data alone; it validates and persists the final state in the database.

5. Data Flow



Core data flow example

- User signs in with Google.
- User creates a party and adds items.
- Each item has a price and note.
- Participants are assigned either ALL or PARTIAL split logic.
- The backend builds a consumption matrix between participants and items.
- The system computes each payer's share and resulting debt-to-settle.
- The final confirmed party is stored and shown in history.

6. ER Diagram

User

- id
- googleId
- email
- username
- avatar
- currencySymbol
- createdAt
- updatedAt

|

| 1 —< many

v

History

- id
- userId
- partyId
- createdAt

Party

- id
- name
- date
- totalAmount
- slipUrl
- createdAt
- updatedAt

|

| 1 —< many

+-----+

Item	
- id	
- name	
- price	
- note	
- partyId	

+-----+

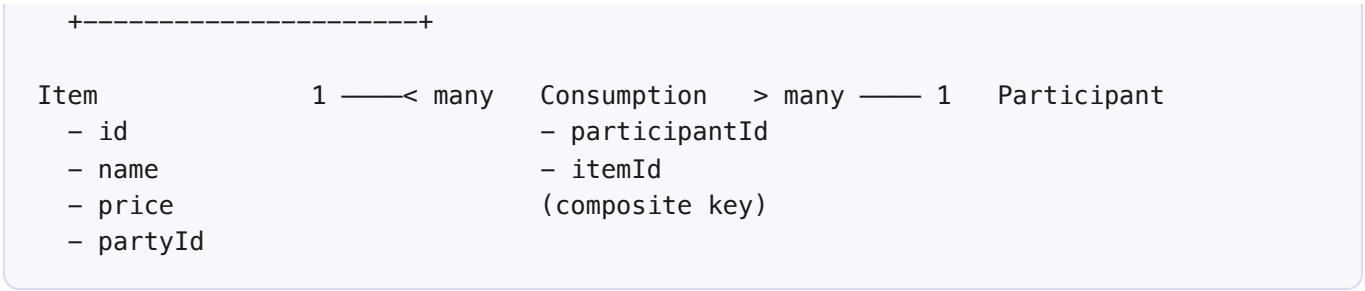
Party

|

| 1 —< many

+-----+

Participant	
- id	
- name	
- splitType	
- partyId	



Relationship summary

- User to History: one-to-many
- User to Party is indirect through History
- Party to Item: one-to-many
- Party to Participant: one-to-many
- Item to Consumption: one-to-many
- Participant to Consumption: one-to-many
- Consumption links item and participant in a many-to-many relationship resolved through a composite primary key

7. Application Logic Summary

The main business logic of the app revolves around splitting shared costs fairly.

- Item cost is stored as a monetary value.
- Participant split mode determines whether they pay for all or only selected items.
- Consumption records define which participant consumed which item.
- Final totals are calculated from these relationships.
- The application presents a summary that indicates who paid, who owes, and what share each member should receive.

8. Deliverable Status

This project is structured as a practical full-stack application suitable for:

- learning full-stack architecture
- building a real demo app
- demonstrating API + database + frontend integration
- showcasing portfolio-level engineering work

It is not intended to be a production-grade payment or settlement service, but it demonstrates the full lifecycle of a real application: UI, server, auth, validation, persistence, and calculation logic.

9. Key Takeaway

Cleft addresses the real-world problem of splitting expenses among a group without relying on manual calculations and inconsistent notes. The project combines frontend interaction, backend logic, and database persistence into one cohesive workflow and serves as a strong example of full-stack development for internship/portfolio purposes.